

Experiment # 03

Basic Shell scripting using gedit/vi

Objective

- Vi Introduction
- Setting access permission of files
- Executing shell script
- User and System Variables
- Shell Arithmetic
- Exit status

In this lab you are introduced to shell programming, how to write script, execute them etc. We will get started with writing small shell script that will print "Knowledge is Power" on screen. Before starting with this you should know. How to use text editor such as vi.

Software Tools

- Ubuntu 16.04
- GCC Compiler
- https://www.tutorialspoint.com/execute_bash_online.php

Theory

Introduction to Vi

Under Linux, there is a free version of Vi called Vim (Vi Improved). Vi (pronounced vee-eye) is an editor that is fully in text mode, which means that all actions are carried out with the help of text commands. This editor, although it may appear of little practical use at first, is very powerful and can be very helpful in case the graphical interface malfunctions.

Syntax:

- vi name_of_the_file
- Once the file is open, you can move around by using cursors.
- Press I to switch to insert mode.
- Press escape key to switch to command mode

Basic commands:

Command	Description
:q	Quit the editor (without saving)
:q!	Forces the editor to quit without saving (even if changes were made to the document)
:wq	Saves the document and quits the editor

:filename Saves the document under the specified name

To search a string forwards in the file, say "myword", type:
:/myword

How to write shell script:

Following steps are required to write shell script:

- ✓ Use any editor like vi or gedit to write shell script.
- ✓ To close vi window press : and then given command (i.e. q, !q or wq)
- ✓ To save and close gedit window press ALT+F4
- ✓ After writing shell script set execute permission for your script as follows

Execute your script as

Now you are ready to write first shell script that will print "Knowledge is Power" on screen.

```
$ vi first
#
# My first shell script
#
clear
echo "Knowledge is Power"
```

- After saving the above script, you can run the script as follows:
\$./first
- This will not run script since we have not set execute permission for our script *first*; to do this type command
\$ chmod 755 first
\$./first
- First screen will be clear, then Knowledge is Power is printed on screen.
- To delete the character that is currently under the cursor you must press x (and you must not be in *Insert* mode because if you are you will insert the character x instead of deleting the character under the cursor). So if you have just opened the file with vi, you can immediately use x to delete characters. If you are in *Insert* mode you have to leave it first with *ESC*.
- If you have made changes and want to save the file, press :x (again you must not be in *Insert* mode. If you are, press *ESC* to leave it).

Script Command(s)	Meaning
\$ vi first	Start vi editor
#	# followed by any text is considered as comment. Comment gives more information about script, logical
# My first shell script	
#	

	explanation about shell script. <i>Syntax:</i> # comment-text
Clear	clear the screen
echo "Knowledge is Power"	To print message or value of variables on screen, we use echo command, general form of echo command is as follows <i>syntax:</i> echo "Message"

Variables in Shell

To process our data/information, data must be kept in computers RAM memory. RAM memory is divided into small locations, and each location had unique number called memory location/address, which is used to hold our data. Programmer can give a unique name to this memory location/address called memory variable or variable (Its a named storage location that may take different values, but only one at a time).

In Linux (Shell), there are two types of variable:

- (1) **System variables** - Created and maintained by Linux itself. This type of variable defined in CAPITAL LETTERS.
- (2) **User defined variables (UDV)** - Created and maintained by user. This type of variable defined in lower letters.

You can see system variables by giving command like **\$ set**, some of the important System variables are: HOME, USER

NOTE that some of the above settings can be different in your PC/Linux environment. You can print any of the above variables contains as follows:

```
$ echo $USERNAME
```

```
$ echo $HOME
```

```
$ echo -e "Geeks \bfor \bGeeks" # this command eliminates all the spaces in the text (-e)
```

How to define User defined variables (UDV)

To define UDV use following syntax at run time

Syntax:

```
read variable_name1 variable_name2
```

To define UDV use following syntax

Syntax:

```
variable_name=value
```

'value' is assigned to given 'variable name' and Value must be on right side = sign.

Example:

```
$ no=10# this is ok
```

```
$ 10=no# Error, NOT Ok, Value must be on right side of = sign.
```

To define variable called 'vech' having value Bus

```
$ vech=Bus
```

To define variable called n having value 10

```
$ n=10
```

Rules for Naming variable name (Both UDV and System Variable)

(1) Variable name must begin with Alphanumeric character or underscore character (_), followed by one or more Alphanumeric character. For e.g. Valid shell variable are as follows

(2) Don't put spaces on either side of the equal sign when assigning value to variable. For e.g. In following variable declaration there will be no error

```
$ no=10
```

But there will be problem for any of the following variable declaration:

```
$ no =10
```

```
$ no= 10
```

```
$ no = 10
```

(3) Variables are case-sensitive, just like filename in Linux. For e.g.

```
$ no=10
```

```
$ No=11
```

```
$ NO=20
```

```
$ nO=2
```

Above all are different variable name, so to print value 20 we have to use \$ echo \$NO and not any of the following

```
$ echo $no # will print 10 but not 20
```

```
$ echo $No# will print 11 but not 20
```

```
$ echo $nO# will print 2 but not 20
```

(4) You can define NULL variable as follows (NULL variable is variable which has no value at the time of definition) For e.g.

```
$ vech=
```

```
$ vech=""
```

Try to print it's value by issuing following command

```
$ echo $vech
```

Nothing will be shown because variable has no value i.e. NULL variable.

How to print or access value of UDV (User defined variables)

To print or access UDV use following syntax

Syntax:

```
$variablename
```

Define variable vech and n as follows:

```
$ vech=Bus
$ n=10
$ echo $vech
$ echo $n
```

Caution: Do not try **\$ echo vech**, as it will print vech instead its value 'Bus' and **\$ echo n**, as it will print n instead its value '10', You must *use \$ followed by variable name*.

```
$ vi variscript
#
#
# Script to test MY knowledge about variables!
#
myname=Vivek
myos = TroubleOS
myno=5
echo "My name is $myname"
echo "My os is $myos"
echo "My number is myno, can you see this number"
```

Shell Arithmetic

Use to perform arithmetic operations.

Syntax:

```
expr op1 math-operator op2
```

Examples:

```
$ expr 1 + 3
$ expr 2 - 1
$ expr 10 / 2
$ expr 20 % 3
$ expr 10 \* 3
$ echo `expr 6 + 3`
```

Note:

expr 20 %3 - Remainder read as 20 mod 3 and remainder is 2.

expr 10 * 3 - Multiplication use * and not * since its wild card.

For the last statement note the following points

(1) First, before expr keyword we used ` (back quote) sign not the (single quote i.e. ') sign. Back quote is generally found on the key under tilde (~) on PC keyboard OR to the above of TAB key.

(2) Second, expr is also end with ` i.e. back quote.

(3) Here `expr 6 + 3` is evaluated to 9, then `echo` command prints 9 as sum

(4) Here if you use double quote or single quote, it will NOT work

For e.g.

\$ echo "expr 6 + 3" # It will print `expr 6 + 3`

\$ echo 'expr 6 + 3' # It will print `expr 6 + 3`

\$ echo `expr 6 + 3` # It will print 9

More about Quotes

There are three types of quotes

Quotes	Name	Meaning
"	Double Quotes	"Double Quotes" - Anything enclosed in double quotes removed meaning of that characters (except \ and \$).
'	Single quotes	'Single quotes' - Enclosed in single quotes remains unchanged.
`	Back quote	`Back quote` - To execute command

Example:

echo "Today is date"

Can't print message with today's date.

echo "Today is `date`"

It will print today's date as, Today is Tue Jan, Can you see that the ``date`` statement uses back quote?

Exit Status

By default in Linux if particular command/shell script is executed, it returns two types of values which are used to see whether command or shell script executed is successful or not.

(1) If return value is zero (0), command is successful.

(2) If return value is *nonzero*, command is not successful or some sort of error executing command/shell script.

This value is known as ***Exit Status***.

But how to find out exit status of command or shell script?

Simple, to determine this exit status you can use `$?` special variable of shell. For e.g. (This example assumes that **unknown1file** does not exist on your hard drive)

\$ rm unknown1file

It will show error as follows

rm: cannot remove `unkowm1file': No such file or directory
and after that if you give command

\$ echo \$?

it will print nonzero value to indicate error. Now give command

\$ ls

\$ echo \$?

It will print 0 to indicate command is successful.

Lab Tasks

NOTE: *Do all the task in TERMINAL/vi/gedit and execute files.*

1. **Simulate** following commands in Linux and note down the exit status:
 - a. `$ expr 1 + 3`
 - b. `$ echo $?`
 - c. `$ echo Welcome`
 - d. `$ echo $?`
 - e. `$ wildwestcanwork?`
 - f. `$ echo $?`
 - g. `$ date`
 - h. `$ echo $?`
 - i. `$ echon $?`
 - j. `$ echo $?`
2. **Simulate** command/commands in Linux to define variable x with value 10 and print it on screen. Check exit status.
3. **Simulate** command/commands in Linux to define variable xn with value *Hello* and print it on screen. Check exit status.
4. **Simulate** command/commands in Linux to print sum of two numbers, let's say 6 and 3?
5. **Simulate** command/commands in Linux to define two variable x=20, y=5 and then to print division of x and y (i.e. x/y)
6. **Simulate** command/commands in Linux to modify above and store division of x and y to variable called z
7. **Simulate** command/commands in gedit to make a simple calculator in which addition, multiplication, subtraction and division on two user defined variables is calculated.
8. **Simulate** command/commands in gedit that contains today's date in script, execute the program and display its output.
9. **Simulate** command/commands in gedit that contains contents of your home directory in script, execute the program and display its output.
10. **Simulate** command/commands in gedit that contains your present work directory in script, execute the program and display its output.

11. **Simulate** the following commands in Linux and state which of the command is correct and why for printing your home directory location,

a) \$ echo \$HOME

OR

(b) \$ echo HOME

Conclusion:

Lab Instructor
Fasih Ahmad